Finding Bugs in Python Native Extensions via Safety Obligation Checking

***Abstract*—Python native extensions written in C/C++ are widely used to provide computational acceleration, hardware interaction, and system-level functionality for performance-critical Python libraries. Developing such extensions requires complying with the safety requirements of the Python/C API, including reference ownership management, null-value handling, exception propagation, and resource lifecycle management. However, these requirements are dispersed throughout extensive CPython documentation and are difficult to reason about across complex execution paths, causing bugs such as memory leaks, use-after-free vulnerabilities, and interpreter crashes that often elude conventional static analysis. We make the key observation that Python/C API safety requirements can be systematically modeled as *dischargeable safety obligations*. Based on a systematic study of CPython documentation and hundreds of real-world bugs, we derive 13 obligation categories that capture the dominant classes of Python/C API misuse and formally define the conditions under which each obligation is discharged. We propose PyCGuard, a bug detection framework for Python native extensions based on safety-obligation checking. PyCGuard first employs lightweight static analysis to identify obligation-generating API usages and eliminate candidates lacking evidence of violations. It then uses structured LLM reasoning to inspect the retained paths and report cases where no admissible discharge condition is identified. We evaluate PyCGuard on 9 widely used open-source projects. PyCGuard finds 86 previously unknown bugs with 82.7% precision, of which 45 have been confirmed or fixed by maintainers.**

***Index Terms*—Python native extensions, Python/C API, static analysis, large language models, bug finding**

I. Introduction

Python native extensions connect Python programs to C and C++ code. This mechanism supports numerical kernels, device runtimes, operating-system interfaces, and other functionality that cannot be implemented efficiently in pure Python. The same boundary removes native code from the memory and error management provided by the interpreter. Extension developers must instead follow the Python/C API rules for reference ownership, nullable returns, exception state, allocator domains, buffers, garbage collection, and thread state [1], [2]. A violation can leak an object, access a released value, corrupt interpreter state, or terminate the process that hosts the extension.

These rules are difficult to enforce for two reasons. First, the relevant information is distributed across the Python/C API reference manual. The correct action depends on details such as whether an API returns a new or borrowed reference, steals an argument, preserves an exception, or pairs with a specific release operation. Second, compliance is path-sensitive. A newly created object can be returned on one path, transferred to a container on a second path, and require an explicit decrement on every remaining path. A cleanup label that is correct for an owned reference can release a borrowed reference on another branch. The defect is therefore not characterized by an isolated call or a fixed API pair.

Prior Python/C analyzers make important progress on reference-count errors. Pungi transforms reference-count behavior into affine programs [3], while PyRefcon tracks both reference counts and object lifecycle states with symbolic execution [4]. These analyses target reference management and require detailed models of API and program behavior. They do not provide a common representation for the broader set of Python/C API requirements, including error-state discipline, allocator agreement, buffer lifetime, numeric conversion, garbage-collection protocols, and capsule metadata. General native-code analyzers cover some low-level defects but lack the ownership and interpreter-state semantics that determine whether a Python/C API use is correct.

Large language models offer a complementary ability to reason about source code and natural-language specifications. Unconstrained bug finding, however, asks a model to infer the relevant property, identify the program object, trace the paths, and judge the violation in one step. This formulation makes reports difficult to audit and exposes the result to unsupported bug hypotheses. Recent hybrid systems improve practical static analysis with model reasoning [5], and specification-guided systems decompose external requirements before checking code [6], [7]. The open question for Python native extensions is how to formulate the API-specific checking task so that static analysis and model reasoning have separate, auditable roles. Unlike a typestate checker, PyCGuard does not build a complete transfer function for the whole program; unlike a pattern-based checker, obligation definitions are separated from syntactic suspicion; and unlike unconstrained LLM bug finding, the model is constrained to a fixed API-induced property with enumerated safe alternatives.

We make the observation that a Python/C API requirement can be represented as a *safety obligation* together with explicit *discharge conditions*. An API use creates an obligation for a program object or runtime state. The obligation is safe only when every relevant path satisfies an admissible discharge condition. For example, a new reference satisfies its ownership obligation when the path releases it, transfers it to an API that steals ownership, or returns it to the caller. This representation states what must hold without prescribing one program-analysis technique.

Based on this observation, we develop PyCGuard. An offline construction process extracts normative requirements and correct usage alternatives from CPython documentation and real bug cases. The resulting specification contains 13 obligation categories and 41 discharge conditions. At detection time, lightweight static analysis finds Python/C API call sites, binds them to obligations, and collects the associated program objects. Fifteen syntactic patterns implemented by the static analyzer prune obligation instances that lack a suspicious shape. These patterns decide which obligation instances reach the LLM, trading recall risk for cost reduction; they are not part of the obligation semantics. For each retained instance, a large language model receives the enclosing function, the obligation, and its discharge conditions. The model identifies the trigger, enumerates relevant exits, checks each condition, and returns a structured verdict and evidence.

We evaluate PyCGuard through three research questions. The in-the-wild study audits 104 high-confidence reports from widely used Python extension projects. Eighty-six reports are real previously unknown defects, yielding 82.7% precision, and 45 findings were confirmed by project maintainers. The baseline study reproduces PyRefcon and audits 80 deduplicated findings; it analyzes why PyRefcon achieves only 7.5% precision and examines the five PyRefcon-only true positives to expose the boundary between Python/C API obligations and generic C memory safety. Finally, the design study measures the cost reduction from static pattern-based pruning and ablates the chain-of-thought discharge step and the discharge-condition list to quantify their individual contributions to bug recall.

This paper makes the following contributions:

- We introduce the *safety obligation* abstraction for Python/C API correctness, which captures each API requirement as an obligation with explicit discharge conditions, extending the checkable scope beyond reference counting to exception state, resource protocols, type agreement, numeric bounds, and more.
- We propose PyCGuard, to our knowledge the first Python native extension bug detector grounded in safety-obligation checking, which decomposes the checking task into static obligation binding and structured LLM discharge reasoning to deliver auditable, path-oriented verdicts without requiring a complete program model.
- We evaluate PyCGuard on nine widely used Python native extensions and find 86 previously unknown bugs at 82.7% precision—more than 10× the precision of the closest reproducible baseline—of which 45 have been confirmed or fixed by maintainers.

II. Background and Motivation

A. Python Native Extensions

CPython exposes a C interface for defining modules, types, and functions and for manipulating interpreter objects. A native function commonly parses Python arguments, creates or accesses `PyObject` values, calls other Python/C APIs, and returns either a Python object or an error indicator. The interface uses conventions rather than a C type system to encode many safety properties. A *new reference* transfers ownership to the caller, while a *borrowed reference* remains owned elsewhere. Some APIs steal ownership of an argument. Most object-producing APIs return `NULL` on failure and set the thread-local exception indicator. Other APIs use integer sentinels, and several resource APIs require a matching release operation [1].

The conventions interact. If `PyTuple_New` returns `NULL`, native code must avoid dereferencing the result, preserve the pending exception, and release objects acquired earlier on that path. If `PyTuple_SetItem` succeeds, it steals an item reference, so a later decrement can access a freed object. Correctness therefore depends on the API contract, the program object to which the contract applies, and the path between acquisition and exit.

B. Python/C API Safety Requirements

We call a requirement *API-induced* when a Python/C API call or a Python-runtime callback creates the relevant duty. Reference ownership is the most visible example, but the same structure appears in other parts of the interface. A successful `PyObject\_GetBuffer` call creates a duty to call `PyBuffer\_Release`; a fallible conversion creates a duty to check its sentinel before use; releasing the global interpreter lock creates a duty to reacquire it before another Python/C API call; and a garbage-collected type creates a multi-part lifecycle duty for allocation, tracking, traversal, clearing, and deallocation.

Three properties make these requirements unsuitable for simple call-pair matching. A requirement can admit several correct outcomes, such as release, transfer, or return. The appropriate outcome can differ across paths. In addition, a syntactically present action can be semantically wrong, as when a cleanup block decrements a borrowed reference or frees memory with an allocator from another domain. These properties motivate an explicit representation of both the duty and its admissible discharge conditions.

C. Motivating Example

Figure 1 presents a defect found in the SciPy `fitpack\_sphere` wrapper. The function receives `tt` and `tp` from `PyArg\_ParseTuple`. When `iopt == -1`, it assigns these borrowed references to `ap\_tt` and `ap\_tp`. A later validation failure reaches the shared cleanup block, which decrements both variables. The cleanup is correct when `iopt == 0` because that branch creates new arrays, but it is unsafe for the borrowed-reference branch. The decrement can prematurely release an object still owned by the caller.

A source-level pattern such as “assignment followed by decrement” cannot distinguish the branches. Tracking only the final reference-count delta is also insufficient unless the analysis recovers the ownership semantics of both assignments and the feasibility of each cleanup path. Under our formulation, the two assignments bind a reference-ownership obligation. The new-reference branch is discharged by cleanup on failure and

```
if (iopt == -1) {
    ap_tt = tt; // borrowed
    ap_tp = tp; // borrowed
} else if (iopt == 0) {
    ap_tt = PyArray_SimpleNew(...); // new
    if (ap_tt == NULL) goto fail;
    ap_tp = PyArray_SimpleNew(...); // new
    if (ap_tp == NULL) goto fail;
}
if (invalid_dimensions) goto fail;
...
fail:
Py_XDECREF(ap_tt);
Py_XDECREF(ap_tp);
return NULL;
```

Fig. 1. Path-dependent ownership in `fitpack\_sphere`. The cleanup releases owned arrays created by the second branch but also releases borrowed arrays assigned by the first branch.

ownership transfer on success. The borrowed-reference branch has no owned reference to release, and the decrement violates the post-acquisition condition. The example illustrates why the checker must combine API semantics with path-specific discharge reasoning.

III. Safety Obligation Construction

A. Overview

The purpose of obligation construction is to turn dispersed API prose and bug evidence into a reusable checking specification. Construction occurs offline and is independent of a target project. Each specification entry records a trigger, the safety property induced by the trigger, the program subject, and the conditions that discharge the property. Detection then instantiates these entries at concrete API calls, as described in Section IV.

The specification contains only semantic information: a safety obligation and the conditions that discharge it. Operational optimizations are deliberately kept outside this representation. For example, the absence of a nearby `Py\_DECREF` cannot define an ownership violation because the reference may instead be transferred or returned. Section IV introduces such syntactic patterns as part of static pruning.

B. Evidence Sources

The primary source is the CPython 3.14 Python/C API reference manual [1]. We inspect chapters that specify object ownership, reference counting, exception handling, memory allocation, buffer access, thread state, garbage-collection support, container iteration, argument conversion, and capsules. Normative statements identify the trigger and the required property. Notes and examples provide admissible safe outcomes. We also inspect real Python/C API bug-fix commits. We mined 442 fix commits from 23 open-source Python extension repositories via the GitHub API, filtering to commits that modify C or C++ source files and whose messages or diffs reference Python/C API constructs; 395 of the 442 commits involve CPython FFI code. These cases expose requirements that are easy to overlook in documentation, interactions between obligations, and recurring code forms suitable for pruning patterns. We stopped adding obligation categories after three consecutive documentation sections and ten consecutive bug cases introduced no new obligation type. The empirical studies by Yang and Cai and by Hu and Zhang provide independent corroboration that ownership, exception, and compatibility mistakes are the dominant failure classes in real-world cross-language code [2], [8].

We use bug cases to refine and validate documented requirements, not to infer correctness from frequency. A recurring idiom is not considered safe when it contradicts the API documentation. Conversely, an unusual idiom can discharge an obligation when its path and ownership semantics satisfy the documented property.

C. Construction Procedure

The construction procedure has three steps. First, we extract a candidate requirement and record its triggering APIs, subject, failure mode, and cited documentation. For the reference-counting rule that a new reference belongs to its caller, for example, the trigger is a new-reference-returning API, the subject is its result, and the failure modes include leaks and premature release.

Second, we normalize requirements by the safety property rather than by API name. APIs that create the same duty share one obligation even when their surface signatures differ. Requirements remain separate when they require different evidence. Reference ownership and null validation can therefore be bound to the same call without collapsing their meanings.

Third, we derive discharge conditions from documented safe alternatives and validated fixes. Conditions are intentionally sufficient rather than exhaustive. An owned reference, for example, can be released on every exit, transferred to an ownership-consuming API, or returned to the caller. A condition records the semantic outcome, not one exact syntax. We merge conditions that establish the same outcome and retain separate conditions when they require different reasoning.

D. Safety Obligations

Table I summarizes the resulting specification. The 13 categories cover reference lifecycle, failure handling, native resources, type and value agreement, and runtime protocols. A single API can bind several categories. For example, an object-producing call can create ownership, null-validation, cleanup, and exception-state obligations. Conversely, one category can cover many APIs that share a contract.

E. Discharge Conditions

Let o be an obligation instance bound in function f, let $D(o)$ be its discharge conditions, and let $\Pi(o, f)$ be the relevant paths from its trigger to an exit or unsafe use. For the default disjunctive mode, the intended judgment is

$$Safe(o, f) \equiv \forall \pi \in \Pi(o, f),\ \exists d \in D(o) : d(\pi). \qquad (1)$$

Thus, different paths can satisfy different conditions. The GC lifecycle obligation uses a conjunctive mode because

TABLE I

PYTHON/C API SAFETY OBLIGATION TYPES AND THEIR DISCHARGE CONDITIONS.

Obligation type	Obligation description	Discharge condition	Discharge-condition description
Reference ownership	Resolve every owned reference and prevent reuse after ownership transfer.	**Explicit release**	Release the reference on every exit path.
		Ownership transfer	Pass it to an API that assumes release responsibility.
		Return to caller	Return it as the function's new-reference result.
		Post-steal non-use	Do not use or release it after a steal unless first retained.
Reference validity	Keep a borrowed reference valid at every use.	**Scope-safe use**	Finish all uses before any operation that can invalidate it.
		Strong promotion	Retain it before invalidation or persistent storage.
Null/error validation	Prove a fallible result valid before unsafe use.	**Dominating check**	Check NULL or the error sentinel before every unsafe use.
		Infallible source	Obtain the value from a source infallible for this input.
		NULL-safe use	Use the value only in NULL-tolerant operations.
Error propagation	Handle or propagate failures and clean resources on failing paths.	**Cleanup block**	Release acquired resources and return an error indicator.
		Early return	Release resources inline before returning the error.
		Recovery	Clear the exception and execute an intentional fallback.
		NULL-safe cleanup	Initialize cleanup variables and use NULL-safe releases.
Exception state	Resolve pending exceptions before normal API work and preserve prior state.	**Immediate check**	Inspect exception state before a non-safe API call.
		Safe-only work	Perform only exception-safe operations while one is pending.
		Prompt resolution	Clear and recover, or return an error, before normal work.
		Dealloc preservation	Save and restore pre-existing state around finalization.
Resource protocol	Pair each acquisition with its correct release and allocator domain.	**Matched release**	Apply the matching release on every successful-acquire path.
		Domain consistency	Free memory through the allocator domain that created it.
		Failure-aware skip	Skip release when acquisition failed and returned NULL.
		Safe reallocation	Preserve the original pointer until reallocation succeeds.
Post-release safety	Prevent repeated release, later access, and reentrant inconsistent state.	**NULL assignment**	Clear the pointer immediately after releasing it.
		No later access	Do not read, dereference, or pass the released resource.
		Container-safe update	Use Py_CLEAR or Py_SETREF for observable members.
		Local-only scope	Ensure reentrant code cannot observe the released object.
		Buffer-alias lifetime	Finish alias reads before releasing the underlying buffer.
Type agreement	Match API formats with C types, widths, encodings, and protocols.	**Format-type match**	Match each format code to the corresponding C argument type.
		Platform width	Use format codes consistent with platform-dependent widths.
Numeric and bounds safety	Prevent narrowing, allocation overflow, and out-of-bounds access.	**Range check**	Check the range before narrowing or indexed access.
		Allocation check	Check size arithmetic for overflow before allocation.
		Sufficient type	Store the value in a type wide enough for the platform.
Thread safety	Use the Python/C API under a valid interpreter-lock context.	**Valid context**	Hold the GIL whenever invoking the Python/C API.
		Blocking-I/O release	Release the GIL for blocking work and reacquire it first.
GC lifecycle	Complete allocation, tracking, traversal, clearing, and deallocation.	**GC allocation domain**	Use matching GC-aware allocation and deallocation.
		Tracking protocol	Track after initialization and untrack before field clearing.
		Traverse and clear	Visit every owned field and clear it with Py_CLEAR.
Iteration integrity	Do not structurally mutate a container during iteration.	**Read-only body**	Perform no direct or indirect mutation while iterating.
		Iteration copy	Iterate over a copy when the original may be mutated.
Capsule name stability	Keep a capsule name valid and consistent throughout its lifetime.	**String literal**	Use an immutable string literal with static lifetime.
		Static storage	Store the name in a static or module-lifetime constant.
		Destructor-managed	Free a dynamic name only in the capsule destructor.

allocation, tracking, traversal, clearing, and deallocation are complementary aspects of one garbage-collection protocol. For this category, all applicable conditions must hold.

Equation 1 defines the question asked by the checker. It does not claim that the LLM enumerates paths soundly or proves the equation. The structured output records the paths and condition judgments used to reach a verdict so that a reviewer can inspect the reasoning.

The ownership obligation in Figure 1 has four conditions: explicit release, ownership transfer, return to the caller, and no use after a steal. On each failure path of the new-reference branch, explicit release holds. On the successful branch, returned arrays transfer ownership. The borrowed-reference branch does not create an owned reference, so its cleanup must not apply the release action. Binding ownership semantics to the concrete assignment exposes this mismatch.

The same representation applies beyond ownership. Null validation can be discharged by a dominating check, an infallible source for the given input, or exclusively null-tolerant uses. A resource protocol requires matched release, allocator-domain agreement, and a failure-safe reallocation pattern. Capsule names can use literal, static, or destructor-managed storage. These conditions specify semantic outcomes rather than one cleanup-label name, macro, or branch shape, allowing equivalent C, C++, and generated-wrapper idioms.

F. *Traceability and Evolution*

Each obligation entry retains its trigger API categories, discharge conditions, and documentation locations in one versioned JSON record. This structure supports two audits. A specification audit can trace a property or condition back to the CPython reference chapter that motivates it. A detection audit can trace a report from a concrete call site to the bound SO, the evaluated DCs, and the final path evidence. Keeping both links is necessary because a correct API rule can still be bound to the wrong subject, and a correct binding can still receive an incorrect path judgment.

The specification is designed to evolve with CPython. A new API that follows an existing ownership or error convention usually adds a trigger without creating a new SO. A new semantic requirement adds a condition or category only when the existing vocabulary cannot express its safe outcomes. Static pruning patterns are versioned separately in the analyzer, so refining a heuristic does not change the meaning of past obligations.

IV. DETECTION PIPELINE

A. *Overview*

Figure 2 presents the two phases of PYCGUARD. The offline phase constructs the obligation specification described in Section III. The online phase consumes a native-extension project and produces auditable bug reports. Static analysis first discovers candidate functions, binds API calls to obligations, and prunes obligation instances using syntactic patterns. Structured LLM checking then tests every retained obligation against its discharge conditions. Only high-confidence violated judgments become reports.

The pipeline uses the function as its context unit and a pair (f, o) as its checking unit. Grouping calls by function preserves cleanup labels and branch conditions shared by several API calls. Separating obligations prevents one large prompt from asking the model to discover arbitrary defects. A function can therefore produce several checking units before pruning, each with one property and a small set of discharge alternatives.

B. Candidate Generation

a) Source discovery.: The scanner walks C and C++ source and header files. A file is selected when it directly includes `Python.h` or contains Python/C API symbols. The second criterion covers indirect includes and generated wrappers. Header files are recorded because inline extension code can reside in headers, although the evaluation reports source-file coverage separately.

b) Call-site discovery.: Tree-sitter parses each selected file and identifies call expressions and their enclosing function definitions. The current specification indexes 147 API names. Calls outside a function are ignored because the checker requires an enclosing control-flow context. Parse errors are retained as metadata so that malformed or dialect-specific input can be audited.

c) Obligation binding.: For each indexed call, PyC-Guard retrieves all associated SOs and records the API, arguments, location, and obligation category. It extracts the subject from the left-hand side for object-producing APIs, from the consumed argument for steal-reference APIs, and from the first argument for decrement operations. All trigger sites in the same function are aggregated. Repeated calls at the same location are deduplicated, and the union of bound SOs forms the unpruned candidate.

d) Local use facts.: The analyzer traverses the enclosing syntax tree for references to each trigger variable. It classifies simple assignments, null checks, decrement calls, ownership-consuming calls, returns, and other uses. It also follows simple local aliases. These facts support static pattern matching; they are not used to establish a final safe verdict.

C. Static Pattern-Based Pruning

The pruning module defines 15 risk-indicator patterns for eight SO categories; they are analyzer rules rather than fields in the obligation specification. The module maintains a registry from an SO identifier to an executable pattern checker. Each checker consumes the trigger sites, the local-use facts collected for trigger variables, and the enclosing function text. This organization keeps pattern evolution independent of the JSON specification: adding or weakening a screen changes candidate selection but not the property sent to the LLM.

The patterns fall into three operational groups. Ownership and lifetime patterns look for a new reference without a release, transfer, or return; an inline new-reference result that cannot be discharged later; borrowed-reference use; repeated release sites; uninitialized cleanup variables; and mutation of a reused tuple. Validation and error-state patterns detect a nullable result whose first classified use is not a check, a callback without an exception check, tri-state APIs treated as booleans, and APIs that suppress exceptions. Resource and representation patterns cover unmatched allocation/release operations, unchecked typed casts, missing write-back resolution, and narrowing conversions. These are deliberately syntactic approximations. A matched pattern retains an SO for checking; it never establishes a violation or replaces a DC.

For ownership, for example, the screen retains a new reference when it cannot find a release, transfer, or return form. For null validation, it retains a result when the first classified use after creation is not a null check. The subsequent LLM prompt contains the SO and complete DC set, but not a claim that the matched pattern is defective. The checker must still reason about all relevant paths and admissible discharge alternatives.

The screen is intentionally conservative in two ways. An SO without an implemented indicator is retained. If screening removes every SO bound to a function, the implementation restores the original set rather than dropping the function. This fallback prevents a candidate from disappearing solely because the current pattern vocabulary does not describe it, although it also limits the maximum reduction. RQ3 measures the actual reduction and checks whether any known defect loses its (f, o) pair.

D. Structured Discharge Checking

For every retained pair, the checker constructs a prompt with four components: the SO description, its check mode, the complete DC list, and the full enclosing function. We use the full function because exits, cleanup labels, and ownership transfers often lie outside a local statement slice. Line numbers, trigger sites, and variables from static analysis orient the model, but the model must justify its decision using source evidence.

The requested procedure mirrors Equation 1. The model first identifies the triggering call and subject. It then enumerates normal returns, early returns, error jumps, and other relevant exits. For each path, it records whether each applicable DC holds and cites variables and lines. A disjunctive obligation is `SAFE` only when every listed path has at least one satisfied DC. A conjunctive obligation requires every applicable aspect. If a path lacks a discharge, the verdict is `VIOLATED` and the output names the path and failed conditions.

The response schema contains the trigger API, subject, path list, per-condition judgments, verdict, violation detail, and confidence. The JSON structure makes missing paths and unsupported condition claims visible during audit. It does not eliminate model error, which we evaluate through manual review, malformed output, and prompt ablation.

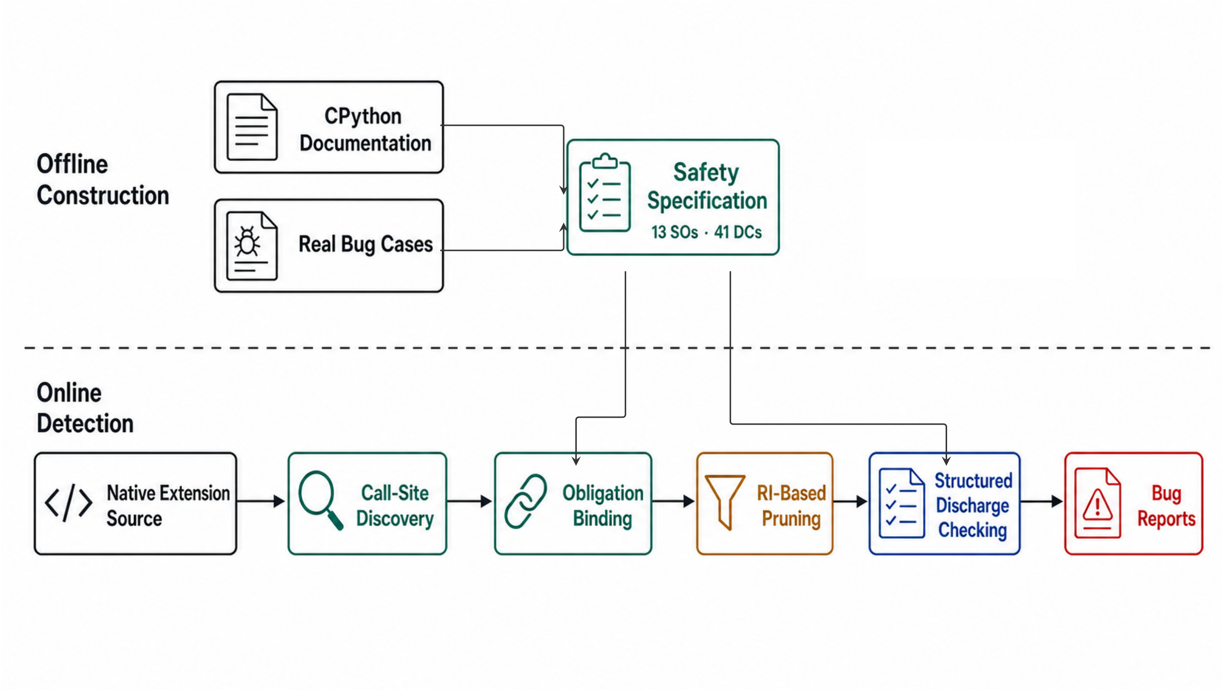

Fig. 2. PyCGuard workflow. Safety obligations and discharge conditions define the checked properties. Static pattern matching prunes bound instances before LLM checking.

a) Multiple triggers and exits.: A function can contain several calls that bind the same SO. The prompt retains all trigger sites and asks the model to identify the subject of each relevant instance. This matters when a cleanup block releases several partially initialized objects or when the same nullable API appears in separate branches. The path record must connect a trigger and subject to the discharge evidence; a decrement of another variable does not satisfy the condition. Normal returns, error returns, cleanup jumps, and fall-through exits are all within scope. The checker does not treat syntactic reachability as proof that a path is feasible.

b) Check ordering.: Retained SO identifiers are sorted before model calls. Stable ordering makes runs easier to compare and gives the early-stop policy deterministic semantics for a fixed model response. Early stop means that the tool reports at most the first violated SO checked for one function in a run. The evaluation therefore counts report units rather than claiming to enumerate every defect effect in a function. Disabling early stop is supported for diagnostic runs that need all SO judgments.

c) Response validation.: The checker accepts only `SAFE` and `VIOLATED` verdicts in the expected JSON object. Network failures are retried with increasing delays. JSON recovery removes limited trailing-comma errors and can recover an explicit verdict from otherwise malformed text; all other responses remain unknown. Recovery status and raw text are preserved so that a parsed verdict does not hide format noncompliance. Malformed and unknown rates are reported separately from detection quality.

E. Report Generation

The default checker stops after the first violated SO in a function. This early-stop policy bounds repeated model calls and gives each report one primary property. Report generation suppresses safe judgments, malformed outputs, and violations below high confidence. A retained report identifies the project, file, function, SO, trigger location, subject, violation path, and model evidence. The high-confidence filter defines the RQ1 audit population; it does not imply that medium- or low-confidence outputs are false.

F. Implementation

PyCGuard is implemented in Python. The static analysis frontend uses the Tree-sitter C and C++ grammars for parsing and AST-based pattern matching. The obligation specification is stored as JSON, from which the tool builds the API index and prompt fragments at load time. The pruning stage applies AST pattern queries to filter obligation instances before any LLM call. The LLM backend is `claude-sonnet-4` at temperature 0.

V. Evaluation

A. Research Questions

We organize the evaluation around three questions:

- RQ1: How effectively does PyCGuard find previously unknown bugs in real Python native extensions?
- RQ2: How does PyCGuard compare with existing Python native-extension bug detectors, and why does it miss bugs found by another detector?

TABLE II
SUBJECTS AND MANUALLY AUDITED HIGH-CONFIDENCE PYCGUARD REPORTS.

Project	Stars	Bug Reports	TP	FP	Precision
SciPy	14.8k	24	20	4	83.3%
pandas	49.1k	18	15	3	83.3%
NumPy	32.3k	15	13	2	86.7%
Pillow	13.6k	15	13	2	86.7%
PyTorch	101k	12	10	2	83.3%
PycURL	1.2k	7	5	2	71.4%
ujson	4.5k	6	5	1	83.3%
TensorFlow	196k	4	3	1	75.0%
psutil	11.2k	3	2	1	66.7%
Total		104	86	18	82.7%

RQ3: How do static pattern-based pruning and structured prompt design affect the cost and bug recall of PYCGUARD?

RQ1 evaluates the end-to-end result after the high-confidence filter. RQ2 compares with the closest reproducible analyzer and studies non-overlapping defects. RQ3 isolates the two design decisions that control analysis cost and model behavior.

B. Subjects and Protocol

We selected nine open-source projects according to three criteria: (1) the project implements substantial performance-critical or system-facing functionality in C or C++; (2) the project is actively maintained with a public issue tracker; and (3) the project is widely adopted, as reflected by GitHub star count and PyPI download volume. Table II reports the quantitative corpus. The scanner manifest contains 489 C/C++ source files from these projects.

a) PyCGuard configuration.: RQ1 uses the configuration in Section IV, including `claude-sonnet-4`, temperature 0, early stop, and the high-confidence report filter. The analysis unit is one project, file, function, and SO tuple. Several unsafe effects in one tuple are counted as one report. We do not use lower-confidence violations to calculate precision. Medium- and low-confidence outputs are suppressed and not audited; all precision claims in this paper are restricted to the 104 high-confidence reports.

b) Manual audit.: For each report, we inspect the complete function, the reported path, the Python/C API contract, and relevant callees or macros when needed. A report is a true positive when the reported behavior violates the claimed SO on a feasible path. A report is a false positive when the path is infeasible, a hidden ownership or cleanup action discharges the SO, or the model misinterprets the API contract. Precision is $TP/(TP+FP)$. Because the study starts from tool reports rather than a complete defect oracle, we do not claim recall for the in-the-wild corpus.

c) Baseline protocol.: We run PyRefcon over all C/C++ source files accepted by its compilation pipeline. We parse 224 HTML warnings and group warnings with the same project, file, function, and bug type into 80 audit units. Each unit is inspected using the reported trace and source context. This deduplication prevents repeated paths for one defect from inflating the result.

TABLE III
ROOT CAUSES OF THE 18 FALSE-POSITIVE PYCGUARD REPORTS.

Root cause	Count
Borrowed, new, or stolen reference semantics	6
Complex control flow	3
Macro expansion	2
C++ RAII or smart-pointer cleanup	2
Interprocedural cleanup	2
State-machine behavior	1
Standard error propagation	1
Infeasible retry path	1

C. RQ1: In-the-Wild Bug Finding

PYCGUARD reports 104 high-confidence findings. Manual audit identifies 86 true positives and 18 false positives, for a precision of $86/104 = 82.7\%$. The true positives span eight checked SO categories. The largest groups involve ownership resolution and null/error validation, but the results also include exception-state errors, resource protocols, post-release safety, type agreement, and numeric bounds. This distribution supports the central scope claim: Python native-extension defects extend beyond reference-count imbalance.

The project-level result is not dominated by one code base. Seven projects contribute at least five true positives, and all nine projects in the audited corpus contain at least two. Of these findings, 45 were accepted by project maintainers, of which 13 have been fixed and 32 are confirmed but awaiting a patch.

The motivating defect in Figure 1 illustrates a common benefit of obligation checking. A shared cleanup block appears locally responsible because it decrements every array variable. The ownership obligation forces the checker to distinguish the borrowed and new-reference branches. Similar true positives require relating a fallible allocation to a later unsafe macro, preserving exception state across callbacks, or pairing a buffer acquisition with all failure exits.

a) False positives.: Table III categorizes the 18 false positives by their immediate root cause. Six require precise knowledge of whether an API borrows, creates, or steals a reference. The next largest group requires reasoning about complex control flow. Macros, C++ destructors, and paired teardown functions hide discharge evidence outside the source form given to the model. These failures suggest that the dominant limitation is missing semantic context rather than a lack of obligation categories.

D. RQ2: Comparison with Existing Tools

PyRefcon is the closest reproducible detector because it directly analyzes Python native code and models Python object lifecycle and reference counts [4]. We run PyRefcon over all

TABLE IV
PYREFCON PER-PROJECT AUDIT RESULT ON THE NINE SUBJECT PROJECTS.

Project	Warnings	Units	TP	FP	Precision
SciPy	76	11	1	10	9.1%
pandas	32	16	4	12	25.0%
NumPy	34	14	1	13	7.1%
Pillow	34	17	0	17	0.0%
PyTorch	×	×	×	×	×
PycURL	26	12	0	12	0.0%
ujson	6	3	0	3	0.0%
TensorFlow	6	2	0	2	0.0%
psutil	10	5	0	5	0.0%
Total	224	80	6	74	7.5%

489 C/C++ source files in the nine projects and manually audit each deduplicated warning unit. Table IV reports the per-project result.

PyRefcon emits 224 raw warnings across 38 of 489 target files; 151 of the remaining files fail to compile in its analysis environment. PyTorch fails entirely because its required CMake-generated headers are absent from the standalone analysis toolchain. After deduplication, 80 audit units remain, of which only 6 are true positives (7.5% precision).

The low true-positive count reflects a fundamental scope mismatch. PyRefcon models only reference-count ownership, so it cannot represent exception-state rules, resource-pairing obligations, type-agreement requirements, numeric bounds, or post-release safety. These five categories account for 35 of PyCGuard's 86 true positives and are entirely outside PyRefcon's scope. Even within reference counting, one of the six PyRefcon true positives overlaps a PyCGuard finding; the remaining five are raw-pointer null dereferences or generic `malloc/free` flows whose correctness depends on project-local C invariants rather than a Python/C API contract, so PyCGuard's static binder never creates an obligation for them.

The 74 false positives reveal the cost of modeling a single lifecycle pattern without explicit discharge alternatives. Fifty-one reports trace a standard `Py\_DECREF` call into the CPython deallocation body and flag it as an ownership release error, treating any decrement as suspect regardless of whether the preceding acquire-release pair is balanced. Eight misread cleanup paths that explicitly destroy an already-allocated object. Nine miss cross-function or cross-iteration invariants, and six arise from macro expansion that hides the actual reference semantics. These patterns are precisely the failure modes that PyCGuard's discharge-condition list is designed to rule out: each discharge condition represents a syntactic form that, when present on every exit path, provides evidence that the obligation is satisfied without requiring interprocedural or macro-expansion reasoning.

We attempted to run CpyChecker using the configuration reported by PyRefcon. The plugin either disabled reference-count verification on GCC 7 and later, failed to initialize with the matching Python headers, or produced no plugin warnings in the usable configuration. We therefore report it as unavailable. Pungi has no public implementation. RID targets general reference-count inconsistencies on the Linux kernel [9] and is not a direct Python/C baseline. CHARON analyzes cross-language vulnerability flows [10] with no public artifact for a same-project run. These tools are discussed qualitatively and do not receive quantitative rows.

E. RQ3: Design Analysis

RQ3 quantifies the value of two design decisions that govern the end-to-end behavior of PYCGUARD. The first is static pattern-based pruning, which decides how many obligations reach the LLM. The second is the obligation prompt, which decides how the LLM reasons about each obligation that it does see. We measure pruning by its marginal effect on checking cost over the 86 manually validated true-positive cases; this controlled replay holds the analyzed function/SO set fixed and therefore isolates the cost contribution of static pruning. We measure the prompt by its effect on bug recall across the same 86 cases.

a) Cost effect of pruning.: Table V reports per-project token usage, dollar cost, and wall-clock time for a controlled replay of the 86 RQ1 true-positive cases with pruning disabled and enabled. The same project/file/function/SO targets are used in both configurations; differences in token usage and time are therefore attributable to pruning rather than to different report populations. Cost uses Anthropic's published Claude Sonnet 4 pricing ($3 / MTok input, $15 / MTok output). The high-confidence filter and the early-stop rule from Section IV are kept enabled in both configurations so that the comparison reflects only the pruning step.

Processing the 86 true-positive cases with pruning disabled costs $2.31 and takes 30.7 minutes of wall-clock LLM time. Enabling pruning cuts this to $1.54 and 21.5 minutes, a reduction of 33.3% in cost and 30.1% in time. The driver is a sharp drop at the binding stage: pruning eliminates between 31% (PyTorch) and 49% (psutil) of bound per-project obligations before any model call, so the LLM never pays the per-call overhead for cases it would ultimately answer SAFE. The end-to-end reduction is smaller than the binding-stage reduction because the obligations that survive pruning are harder on average; the model emits longer structured rationale on them, which cost more output tokens per call. Pruning therefore concentrates LLM effort on syntactically suspicious cases rather than removing work uniformly.

Per-project savings track the fraction of callback-heavy and allocator-only functions that the binders hit but pruning can dismiss. SciPy, NumPy, PycURL, and psutil each achieve at least 33% cost reduction, the highest cuts occurring where short thunk or conversion helpers account for most of the unpruned obligation count. PyTorch shows the smallest reduction (11.8%) because its mixed-language RAII helpers survive pruning at a higher rate; the surviving obligations are in larger, more complex functions, and the structured rationale on them is longer. Even there, every audited true positive is retained,

TABLE V
PER-PROJECT LLM COST ON THE 86 TRUE POSITIVES WITH AND WITHOUT STATIC PRUNING.

Project	Without pruning				With pruning				Reduction (%)
	Input (K)	Output (K)	Cost ($)	Time (min)	Input (K)	Output (K)	Cost ($)	Time (min)	
SciPy	103.0	24.7	0.68	8.0	62.3	17.1	0.44	5.5	-35.3%
pandas	28.4	13.8	0.29	4.4	17.6	9.8	0.20	3.1	-31.0%
NumPy	81.6	15.7	0.48	5.3	47.5	10.4	0.30	3.5	-37.5%
Pillow	37.9	15.0	0.34	4.8	22.7	10.2	0.22	3.2	-35.3%
PyTorch	15.5	8.4	0.17	2.7	11.9	7.4	0.15	2.4	-11.8%
PycURL	10.8	7.4	0.14	2.3	6.0	4.6	0.09	1.4	-35.7%
ujson	6.3	4.7	0.09	1.6	4.1	3.5	0.06	1.2	-33.3%
TensorFlow	3.4	2.7	0.05	0.9	2.4	2.2	0.04	0.7	-20.0%
psutil	9.3	2.4	0.06	0.8	4.8	1.4	0.04	0.4	-33.3%
Total	296.2	94.9	2.31	30.7	179.2	66.7	1.54	21.5	-33.3%

TABLE VI
PER-PROJECT BUG RECALL ON THE 86 TRUE POSITIVES UNDER TWO ABLATIONS.

Project	Target	w/o CoT		w/o DC	
		Found	Recall	Found	Recall
SciPy	20	17 (↓3)	85.0%	19 (↓1)	95.0%
pandas	15	15 (↓0)	100.0%	15 (↓0)	100.0%
NumPy	13	5 (↓8)	38.5%	10 (↓3)	76.9%
Pillow	13	10 (↓3)	76.9%	10 (↓3)	76.9%
PyTorch	10	9 (↓1)	90.0%	10 (↓0)	100.0%
PycURL	5	2 (↓3)	40.0%	1 (↓4)	20.0%
ujson	5	3 (↓2)	60.0%	3 (↓2)	60.0%
TensorFlow	3	1 (↓2)	33.3%	3 (↓0)	100.0%
psutil	2	2 (↓0)	100.0%	2 (↓0)	100.0%
Total	86	64 (↓22)	74.4%	73 (↓13)	84.9%

confirming that the risk indicator set does not discard evidence for any of the audited true-positive cases.

b) Bug recovery effect of CoT prompting and Discharge Conditions.: We next isolate the two reasoning-side design decisions: the chain-of-thought (CoT) discharge procedure encoded in the system prompt, and the per-obligation discharge-condition (DC) list provided in the user prompt. We rerun the 86 RQ1 true positives under two ablated configurations and otherwise hold the pipeline fixed. *w/o CoT* replaces the CoT system prompt with a direct-verdict instruction. *w/o DC* retains the CoT system prompt but removes the DC list from the user prompt; the obligation text alone tells the model what to discharge. The baseline for comparison is the RQ1 configuration (CoT prompt with DC list). The backend, temperature, max tokens, high-confidence filter, and obligation binder are identical across all configurations. We classify a case as *recovered* when the model returns parseable JSON with `verdict=VIOLATED` and `confidence=high`. This ablation is not a precision experiment: because it is run only on independently audited true positives, it measures how often each prompt variant preserves known bug reports; it does not characterize false-positive behavior or deployment precision.

Table VI reports the per-project recovery counts. The RQ1 pipeline queries each function with the specific obligation that the static binder assigned to it, producing one targeted prompt per binding. The ablation harness follows the same binding step but evaluates each of the 86 true-positive cases as an independent unit, without the surrounding context of sibling obligations in the same function. This difference in query scope accounts for the 13-case gap between the end-to-end result (86/86) and the ablation baseline (73/86); both configurations use identical models, prompts, and confidence filters. Removing the CoT system prompt drops recall to 74.4% (64/86). Removing the DC list keeps the total at 84.9% but covers a non-identical set of bugs. The scope difference affects both ablations equally and therefore does not bias the comparison between them.

The CoT step contributes the largest amount of recall. Against the ablation baseline of 73 recovered bugs, removing CoT drops the count to 64, a loss of 9 bugs. These losses concentrate on defect patterns whose discharge depends on enumerating every exit path: reference-count leaks in NumPy and SciPy where the model identifies the decrement on the success path but misses the error-goto branch that leaves the object live; resource leaks in Pillow and SciPy where a paired release is only reached on the normal return; and exception-state violations in TensorFlow where an unchecked call on an alternate branch silently clears the error indicator. In each case, the direct prompt leads the model to declare SAFE after inspecting only the dominant path. The four-step CoT forces an explicit per-path enumeration before the verdict, which is what surfaces these multi-path discharge defects. Conversely, w/o CoT recovers three bugs that the baseline misses, all involving short callbacks with a single unchecked return; direct prompting flags them immediately, while the structured rationale exhausts its reasoning on discharge-condition checks that resolve to SAFE on the only exit path.

The DC list contributes specificity rather than breadth. Removing it shifts the recovered set by four bugs: two losses involve defects where the obligation text alone is broad enough to admit a plausible SAFE answer, and the structured list of discharge conditions is what narrows the model to the specific failure pattern. The two recoveries go in the opposite direction:

when the DC list is absent, the model falls back to a more general violation criterion and flags defects that the structured discharge walk would resolve as SAFE on a feasible path. The net effect on total recall is zero, confirming that the DC list reshapes the checking behavior rather than expanding it.

The two ablations together support the central design claim. The chain-of-thought walk is the primary source of true-positive recovery, recovering 9 more cases than direct prompting on the same fixed set. The discharge conditions sharpen the specificity of each individual check without trading away recall. Removing either component degrades true-positive recovery on this fixed set in a distinct and complementary way, which is consistent with their separate roles in the prompt: CoT governs path coverage, and the DC list governs violation criteria.

c) Answers to the research questions.: For RQ1, PyCGuard finds 86 previously unknown bugs among 104 audited high-confidence reports, with 82.7% precision. For RQ2, PyRefcon finds six true positives among 80 deduplicated units; five non-overlapping findings are generic C memory errors outside the Python/C API obligation scope. For RQ3, static pattern-based pruning cuts the LLM cost by 33.3% and wall-clock time by 30.1% on the 86 true-positive cases while preserving every audited finding, and the prompt ablation shows that removing the CoT step drops recall from 84.9% to 74.4% while removing the discharge-condition list shifts the recovered set without changing the total recall.

VI. Discussion and Threats to Validity

A. Why Safety Obligations Help

Safety obligations give static analysis and LLM reasoning different jobs. Static analysis answers where a documented duty can arise and which program object it concerns. The LLM answers a narrower semantic question: whether the visible paths satisfy one of several explicit discharge conditions. This split avoids requiring a complete handwritten transfer model for every API and project helper, while preventing the model from choosing an arbitrary bug property after reading the code.

The representation also makes reports comparable across surface forms. A reference returned, transferred, or decremented uses different syntax but resolves the same ownership duty. Conversely, a decrement can be correct for a new reference and unsafe for a borrowed one. Encoding the required outcome and its alternatives captures this distinction more directly than an isolated pattern.

B. Role of Static Pruning

Static pattern-based pruning controls which obligation instances reach the LLM rather than defining the safety semantics. A pruned instance is one where the static evidence for a violation is absent; it may still contain a real defect if that evidence is hidden inside a macro, a callee, or a generated wrapper.

C. Scope and Limitations

PyCGuard is scoped to duties induced by the Python/C API. It does not attempt to replace a general C/C++ memory-safety analyzer. The PyRefcon-only defects in RQ2 provide concrete examples: raw allocation and null-dereference invariants without a Python/C trigger never reach obligation binding. Extending the method to project-local APIs would require another specification source and a clear rule for composing local and CPython obligations.

The current context is intraprocedural source text. Function-like macros can hide steal semantics, C++ destructors can hide release, and paired teardown functions can hide ownership transfer. Of the 18 false positives, 2 are caused by macro expansion, 2 by C++ RAII or destructor cleanup, and 2 by interprocedural teardown helpers; the remaining 12 require richer ownership-semantics knowledge to resolve. Preprocessing selected macros, summarizing local helpers, and modeling common RAII wrappers could expose the missing discharge evidence without broadening the property scope.

The structured checker remains an LLM-based judgment. It can omit a path, accept infeasible control flow, or misunderstand an API. Temperature 0 reduces sampling variation but does not provide determinism across hosted model versions. The evidence-bearing schema supports audit, not formal soundness. The high-confidence filter also trades report volume for review cost. The current precision therefore describes the surfaced high-confidence set and does not establish recall over all bugs or all model outputs.

D. Threats to Validity

a) Construct validity.: The main metric is precision over high-confidence reports. This metric reflects developer-facing audit cost but omits lower-confidence outputs and cannot measure missed defects. PyCGuard and PyRefcon also use different report units and filters. We state both units and use the comparison to study precision, scope, and overlap rather than relative recall.

b) Internal validity.: Manual labels can be wrong, particularly for rare failure paths and hidden ownership conventions. The audit examines source context, API documentation, and macros or callees when needed. Maintainer feedback and fixes provide additional evidence for submitted reports, but feedback is not available for every finding. The artifact includes report-level labels and rationales so that independent reviewers can challenge individual decisions.

c) External validity.: The subjects emphasize mature scientific, machine-learning, serialization, networking, imaging, and system libraries. Results may not generalize to small extensions, closed-source code, alternative Python implementations, or binding frameworks that generate most native code. The obligations follow CPython 3.14 documentation. API evolution can add triggers or change conditions, requiring the specification and index to be versioned.

d) Reproducibility.: Hosted model behavior and legacy baseline toolchains can change. We report exact model identifiers and decoding parameters and preserve prompts and raw responses. The PyRefcon and CpyChecker environments are containerized, but compilation still depends on project headers and generated files. The anonymous artifact will include target manifests, containers, raw warnings, deduplication records, and audit labels, together with an explanation for any component that cannot be redistributed.

VII. Related Work

A. Python Native-Extension Analysis

Pungi models Python/C reference-count operations as affine programs [3]; CpyChecker tracks objects and expected counts over GCC paths [11]; and PyRefcon adds lifecycle states, nullity, and C++ monitors [4]. PyCGuard instead covers 13 API obligation categories and represents correctness as alternative discharge conditions. PyRefcon remains the closest reproducible baseline because its lifecycle scope overlaps SO-01, SO-02, and SO-07.

Jinn synthesizes dynamic state-machine checkers for JNI and Python/C rules [12]. Other work characterizes Python/C API evolution and bugs [2], jointly analyzes Python and C with shared abstract domains [13], or diagnoses inefficient boundary interactions [14]. PyCGuard targets statically visible native-side API duties rather than executed violations, language interaction errors, or performance.

General reference-count work uses shallow-alias analysis, compositional verification, path-pair inconsistency, or two-dimensional consistency [9], [15]–[17]. Resource analyses detect path-sensitive leaks, infer acquire/release pairs, track C++ smart-pointer heap protocols, or identify disordered cleanup [18]–[21]. These techniques inform PyCGuard's path and resource reasoning but omit CPython ownership, exception-state, garbage-collection, and API-specific discharge semantics.

B. Cross-Language Interface Analysis

JNI analyses detect resource and typestate errors [22], analyze exceptional paths [23], [24], prune before fine-grained exception checking [25], or infer types across FFI boundaries [26]. They demonstrate the need for native-side models but each targets a narrower family of properties.

Related systems detect type and lifetime errors in JavaScript bindings [27], coordinate garbage collection across components [28], or track polyglot vulnerability flows [10]. PyCGuard checks CPython API conformance even when both trigger and consequence remain in native code.

RustBelt formalizes unsafe abstractions [29]; empirical studies characterize unsafe usage and requirements [30]–[32]; and Rudra, MirChecker, SafeDrop, and subsequent analyses detect unsafe-code patterns and safety bugs [33]–[37]. CPython lacks an equivalent language-level discipline, so PyCGuard recovers heterogeneous protocols from documentation and bugs.

C. LLM-Assisted Bug Detection

Learned vulnerability detectors are sensitive to dataset construction and generalization [38], [39], and standalone LLMs remain unreliable vulnerability reasoners [40]. GPTScan instead combines GPT with program analysis [41], supporting PyCGuard's use of explicit program facts and properties.

Hybrid systems validate static warnings [5], infer taint specifications for whole-repository analysis [42], couple LLMs with bidirectional taint analysis [43], or derive binary-analysis semantics [44]. PyCGuard starts from an audited Python/C obligation vocabulary and delegates only candidate path adjudication to the model.

Data-flow facts can constrain LLM bug judgments [45]; LLMDFA decomposes data-flow reasoning and externally validates path feasibility [46]; DAInfer extracts aliasing specifications from documentation [47]; and KNighter synthesizes static checkers from patches [48]. PyCGuard likewise narrows reasoning, but keeps obligations and pruning rules explicit and auditable.

Safe4U decomposes unsafe Rust contracts before checking wrappers [6], while RFCAudit and RFCParser interpret protocol documents [7], [49]. PyCGuard instead normalizes many API statements and real bugs into reusable obligation types with alternative discharge conditions and separate pruning patterns.

VIII. Conclusion

Python native extensions must satisfy dispersed, path-sensitive Python/C API requirements. PyCGuard represents these requirements as safety obligations with explicit discharge conditions, binds and prunes obligation instances with static analysis, and uses structured LLM reasoning to check the retained paths. In the current audited corpus, PyCGuard finds 86 previously unknown bugs among 104 high-confidence reports, with 82.7% precision. The results support obligation-guided checking as a practical way to broaden native-extension bug detection beyond reference-count imbalance while retaining auditable scope and evidence, leaving complete recall and sound path reasoning to future work.

a) Artifact.: The artifact includes the PyCGuard implementation, the safety obligation specification, evaluation scripts, and a record of the 45 maintainer-confirmed findings. An anonymous repository is available at: https://anonymous.4open.science/r/PLACEHOLDER.

References

[1] Python Software Foundation, *Python/C API Reference Manual, Python 3.14.6*, 2026, accessed: 2026-06-27. [Online]. Available: https://docs.python.org/3.14/c-api/

[2] M. Hu and Y. Zhang, "The python/c api: Evolution, usage statistics, and bug patterns," in *Proceedings of the 27th IEEE International Conference on Software Analysis, Evolution and Reengineering*. IEEE, 2020, pp. 532–536.

[3] S. Li and G. Tan, "Finding reference-counting errors in python/c programs with affine analysis," in *ECOOP 2014: Object-Oriented Programming*. Springer, 2014, pp. 80–104.

[4] X. Ma, J. Yan, H. Zhang, J. Yan, and J. Zhang, "Detecting memory errors in python native code by tracking object lifecycle with reference count," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 2023, pp. 1429–1440.

[5] H. Li, Y. Hao, Y. Zhai, and Z. Qian, "Enhancing static analysis for practical bug detection: An llm-integrated approach," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 474–499, 2024.

[6] H. Li, B. Wang, X. Hu, and X. Xia, "Safe4u: Identifying unsound safe encapsulations of unsafe calls in rust using llms," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 457–480, 2025.

[7] M. Zheng, C. Wang, X. Liu, J. Guo, S. Feng, and X. Zhang, "Rfcaudit: Ai agent for auditing protocol implementations against rfc specifications," in *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 2025, pp. 1221–1233.

[8] H. Yang and H. Cai, "Dissecting real-world cross-language bugs," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, p. FSE058, 2025.

[9] J. Mao, Y. Chen, Q. Xiao, and Y. Shi, "RID: Finding reference count bugs with inconsistent path pair checking," in *Proceedings of the 21st International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, 2016, pp. 531–544.

[10] R. Scholtes, S. Khodayari, C.-A. Staicu, and G. Pellegrino, "CHARON: Polyglot code analysis for detecting vulnerabilities in scripting languages native extensions," in *Proceedings of the 10th IEEE European Symposium on Security and Privacy*. IEEE, 2025, pp. 153–168.

[11] D. Malcolm, "Static analysis of cpython extension code with cpychecker," gcc-python-plugin documentation, accessed: 2026-06-27. [Online]. Available: https://gcc-python-plugin.readthedocs.io/en/latest/cpychecker.html

[12] B. Lee, B. Wiedermann, M. Hirzel, R. Grimm, and K. S. McKinley, "Jinn: Synthesizing dynamic bug detectors for foreign language interfaces," in *Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2010, pp. 36–49.

[13] R. Monat, A. Ouadjaout, and A. Mine, "A multilanguage static analysis of python programs with native c extensions," in *Static Analysis*. Springer, 2021, pp. 323–345.

[14] J. Tan, Y. Chen, Z. Liu, B. Ren, S. L. Song, X. Shen, and X. Liu, "Toward efficient interactions between python and native libraries," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 2021, pp. 1117–1128.

[15] A. Lal and G. Ramalingam, "Reference count analysis with shallow aliasing," *Information Processing Letters*, vol. 111, no. 2, pp. 57–63, 2010.

[16] M. Emmi, R. Jhala, E. Kohler, and R. Majumdar, "Verifying reference counting implementations," in *Tools and Algorithms for the Construction and Analysis of Systems*, ser. Lecture Notes in Computer Science. Springer, 2009, vol. 5505, pp. 352–367.

[17] X. Tan, Y. Zhang, X. Yang, K. Lu, and M. Yang, "Detecting kernel refcount bugs with two-dimensional consistency checking," in *30th USENIX Security Symposium*. USENIX Association, 2021, pp. 2471–2488. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/tan

[18] G. Fan, R. Wu, Q. Shi, X. Xiao, J. Zhou, and C. Zhang, "Smoke: Scalable path-sensitive memory leak detection for millions of lines of code," in *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering*. IEEE, 2019, pp. 72–82.

[19] P. Bian, B. Liang, J. Huang, W. Shi, X. Wang, and J. Zhang, "Sinkfinder: Harvesting hundreds of unknown interesting function pairs with just one seed," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 2020, pp. 1101–1113.

[20] X. Ma, J. Yan, W. Wang, J. Yan, J. Zhang, and Z. Qiu, "Detecting memory-related bugs by tracking heap memory management of c++ smart pointers," in *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 2021, pp. 880–891.

[21] Q. Wu, A. Pakki, N. Emamdoost, S. McCamant, and K. Lu, "Understanding and detecting disordered error handling with precise function pairing," in *30th USENIX Security Symposium*. USENIX Association, 2021, pp. 2041–2058. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/wu-qiushi

[22] G. Kondoh and T. Onodera, "Finding bugs in java native interface programs," in *Proceedings of the 2008 International Symposium on Software Testing and Analysis*. ACM, 2008, pp. 109–118.

[23] S. Li and G. Tan, "Finding bugs in exceptional situations of jni programs," in *Proceedings of the 16th ACM Conference on Computer and Communications Security*. ACM, 2009, pp. 442–452.

[24] ——, "Exception analysis in the java native interface," *Science of Computer Programming*, vol. 89, pp. 273–297, 2014.

[25] ——, "Jet: Exception checking in the java native interface," in *Proceedings of the 2011 ACM International Conference on Object-Oriented Programming Systems Languages and Applications*. ACM, 2011, pp. 345–358.

[26] M. Furr and J. S. Foster, "Checking type safety of foreign function calls," *ACM Transactions on Programming Languages and Systems*, vol. 30, no. 4, pp. 18:1–18:63, 2008.

[27] F. Brown, S. Narayan, R. S. Wahby, D. Engler, R. Jhala, and D. Stefan, "Finding and preventing bugs in javascript bindings," in *Proceedings of the IEEE Symposium on Security and Privacy*. IEEE, 2017, pp. 559–578.

[28] U. Degenbaev, J. Eisinger, K. Hara, M. Hlopko, M. Lippautz, and H. Payer, "Cross-component garbage collection," *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, pp. 1–24, 2018.

[29] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, "Rustbelt: Securing the foundations of the rust programming language," *Proceedings of the ACM on Programming Languages*, vol. 2, no. POPL, pp. 66:1–66:34, 2018.

[30] V. Astrauskas, C. Matheja, F. Poli, P. Muller, and A. J. Summers, "How do programmers use unsafe rust?" *Proceedings of the ACM on Programming Languages*, vol. 4, no. OOPSLA, pp. 1–27, 2020.

[31] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang, "Understanding memory and thread safety practices and issues in real-world rust programs," in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2020, pp. 763–779.

[32] M. Cui, S. Sun, H. Xu, and Y. Zhou, "Is unsafe an achilles' heel? a comprehensive study of safety requirements in unsafe rust programming," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. ACM, 2024, pp. 1–13.

[33] Y. Bae, Y. Kim, A. Askar, J. Lim, and T. Kim, "Rudra: Finding memory safety bugs in rust at the ecosystem scale," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. ACM, 2021, pp. 84–99.

[34] Z. Li, J. Wang, M. Sun, and J. C. S. Lui, "Mirchecker: Detecting bugs in rust programs via static analysis," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2021, pp. 2183–2196.

[35] M. Cui, C. Chen, H. Xu, and Y. Zhou, "Safedrop: Detecting memory deallocation bugs of rust programs via static data-flow analysis," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 4, pp. 1–21, 2023.

[36] B. Qin, Y. Chen, H. Liu, H. Zhang, Q. Wen, L. Song, and Y. Zhang, "Understanding and detecting real-world safety issues in rust," *IEEE Transactions on Software Engineering*, vol. 50, no. 6, pp. 1306–1324, 2024.

[37] Z. Zhuang, Z. Wang, W. Meng, and M. R. Lyu, "Don't panic! finding bugs hidden behind rust runtime safety checks," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2025, pp. 186–200.

[38] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3280–3296, 2022.

[39] B. Steenhoek, M. M. Rahman, R. Jiles, and W. Le, "An empirical study of deep learning models for vulnerability detection," in *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering*. IEEE, 2023, pp. 2237–2248.

[40] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "Llms cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," in *Proceedings of the 45th IEEE Symposium on Security and Privacy*. IEEE, 2024, pp. 862–880.

[41] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, "Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. ACM, 2024, pp. 1–13.

[42] Z. Li, S. Dutta, and M. Naik, "Iris: Llm-assisted static analysis for detecting security vulnerabilities," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=9LdJDU7E91

[43] Y. Ji, H. Cao, and J. He, "Staint: Detecting second-order vulnerabilities in php applications with llm-assisted bi-directional static taint analysis," in *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 2025, pp. 3943–3947.

[44] P. Liu, C. Sun, Y. Zheng, X. Feng, C. Qin, Y. Wang, Z. Xu, Z. Li, P. Di, Y. Jiang, and L. Sun, "Llm-powered static binary taint analysis," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 3, pp. 1–36, 2025.

[45] C. Wang, W. Zhang, Z. Su, X. Xu, and X. Zhang, "Sanitizing large language models in bug detection with data-flow," in *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, 2024, pp. 3790–3805.

[46] C. Wang, W. Zhang, Z. Su, X. Xu, X. Xie, and X. Zhang, "Llmdfa: Analyzing dataflow in code with large language models," in *Advances in Neural Information Processing Systems 37*. Curran Associates, Inc., 2024, pp. 131 545–131 574. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/ed9dcde1eb9c597f68c1d375bbecf3fc-Abstract-Conference.html

[47] C. Wang, J. Zhang, R. Wu, and C. Zhang, "Dainfer: Inferring api aliasing specifications from library documentation via neurosymbolic optimization," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 2469–2492, 2024.

[48] C. Yang, Z. Zhao, Z. Xie, H. Li, and L. Zhang, "Knighter: Transforming static analysis with llm-synthesized checkers," in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. ACM, 2025, pp. 655–669.

[49] M. Zheng, D. Xie, Q. Shi, C. Wang, and X. Zhang, "Validating network protocol parsers with traceable rfc document interpretation," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 1772–1794, 2025.